

AI-Powered CRM Email Automation Platform

Complete Technical Documentation
Architecture · Libraries · APIs · Security · Setup

Version 1.0 | July 2026

FastAPI · Ollama · ChromaDB · Redis · Celery · SQLAlchemy

Table of Contents

- 1. Project Overview
- 2. Core CRM Features
- 3. AI Engine Architecture
- 4. Root Directory Structure
- 5. Libraries & Dependencies
- 6. API Endpoints (AI Router)
- 7. Security Architecture
- 8. Setup & Installation

All sections are self-contained. Refer to the codebase for the latest implementation details.

1. Project Overview

The AI-Powered CRM + Email Automation Platform is a state-of-the-art Customer Relationship Management system combined with a fully modular, provider-agnostic AI Engine. It is designed to provide real-time sales analytics, background email synchronisation via Gmail, and context-aware AI capabilities without locking teams into any single LLM provider.

Built with FastAPI (Python 3.11+), SQLAlchemy, Celery, Redis, ChromaDB, and Ollama, the system is engineered for performance, scalability, and enterprise-grade data security.

2. Core CRM Features

Contacts & Leads

- Manage full customer profiles with timeline, interaction history, and lead scoring.
- AI automatically scores each lead from 1-100 based on engagement signals.

Pipelines & Deals

- Sales pipelines with configurable stages and territory tracking.
- Win/Loss analytics and sales cycle reporting.

Campaigns

- Email campaign management with audience segmentation.
- AI-powered email copy generation for campaigns.

Analytics & Dashboards

- Real-time metrics streamed to the frontend via WebSockets.
- Territory performance, pipeline snapshots, and revenue forecasts.

Gmail Integration

- Secure OAuth2 Gmail connection.
- Background synchronisation of email metadata via Celery tasks.
- Thread tracking and automatic reply generation.

3. AI Engine Architecture

The AI Engine is the centerpiece of this refactoring project. All CRM modules communicate exclusively with the AIEngine facade, which internally routes requests to the configured provider.

The full AI pipeline for a 'Generate Reply' operation:

- 1. Context Aggregation (ContextBuilder)**
Gathers the contact's CRM profile, past deals, activity timeline, and persistent AI memory from the CustomerMemory model.
- 2. RAG Retrieval (KnowledgeBase - ChromaDB)**
Embeds the user query and retrieves semantically relevant chunks from company documents (PDFs, CSVs, DOCX) to ground AI responses in facts.
- 3. PII Sanitization (PIISanitizer)**
Scrubs the final assembled context of sensitive data including Credit Card numbers, Social Security Numbers, and Phone Numbers before any data leaves the CRM boundary.
- 4. Prompt Assembly (PromptManager + Jinja2)**
Injects the sanitized context and user query into a decoupled Jinja2 template file (.jinja2), separating business logic from prompt engineering.
- 5. Semantic Cache Check (SemanticCache - Redis)**
SHA-256 hashes the final prompt and queries Redis. If a cached response is found (TTL: 24 hours), it is returned instantly, saving compute resources.
- 6. LLM Generation (OllamaProvider)**
Sends the prompt to the local Ollama instance. Supports both synchronous generation and async streaming via WebSockets.
- 7. Response Parsing (ResponseParser)**
For structured data tasks (lead scoring, classification), the output is validated against a JSON schema. If it fails, the exact error is fed back to the LLM for automatic self-correction (up to 3 retries).
- 8. Memory Extraction (MemoryManager)**
Post-generation, the interaction is analyzed in the background by a Celery task to update the contact's persistent memory (e.g., communication style, pain points, buying signals).

4. Root Directory Structure

```
backend/
  ai/                                -> Modular AI Engine (ALL AI logic lives here)
    cache/                           -> Redis Semantic Caching (semantic_cache.py)
    context/                         -> ContextBuilder (aggregates CRM data + RAG)
    memory/                         -> MemoryManager (persistent customer preferences)
    parser/                         -> ResponseParser (strict JSON with auto-retries)
    prompts/                        -> PromptManager + Jinja2 template files
      templates/                    -> .jinja2 files for each AI task
    providers/                      -> BaseProvider interface + OllamaProvider
    rag/                            -> ChromaDB KnowledgeBase + DocumentParser
    services/                       -> AIEngine Facade (main entry point)
    utils/                          -> PIISanitizer
  auth/                             -> JWT Authentication, User models
  cache/                            -> Redis client (with MemoryCache fallback)
  config/                           -> Pydantic settings (.env loader)
  crm_email/                        -> Gmail API sync logic
  data/                             -> ChromaDB persistence + uploaded docs
  models/                           -> SQLAlchemy models (CRM, AI Memory, Campaigns)
  routers/                          -> FastAPI endpoints (REST + WebSocket)
  services/                         -> CRM business logic services
  tasks/                            -> Celery app initialization
  websocket/                        -> WebSocket message schemas
  workers/                          -> Celery background workers
  ws_manager/                       -> WebSocket connection manager
  main.py                           -> FastAPI app + router registration
  database.py                       -> SQLAlchemy engine + session factory
  requirements.txt                   -> Python dependencies
```

5. Libraries & Dependencies

Core Framework

- fastapi - Async Python web framework
- uvicorn - ASGI server
- sqlalchemy - ORM and database management
- pydantic - Data validation and settings

AI & Machine Learning

- httpx - Async HTTP client (Ollama communication)
- chromadb - Embedded vector database for RAG
- jinja2 - Prompt template engine
- pypdf2 - PDF document parsing for RAG
- python-docx - DOCX document parsing for RAG

Infrastructure

- redis - Caching layer + Celery broker
- celery - Distributed task queue for background jobs

Authentication & Security

- python-jose - JWT token creation and validation
- passlib - Password hashing
- google-auth - Gmail OAuth2 authentication

Utilities

- alembic - Database migrations
- python-dotenv - Environment variable loading

6. API Endpoints (AI Router)

REST Endpoints (prefix: /api/v1/ai)

GET /health

Returns the current health status of the AI Engine and provider.

POST /classify-email

Accepts {subject, body}. Returns a JSON category object (e.g., {"category": "support"}). Uses ResponseParser with auto-retry.

POST /generate-reply

Accepts {contact_id, email_body, tone}. Returns a fully context-aware email draft incorporating CRM data and company knowledge.

WebSocket Endpoint

WS /ws/stream

Real-time streaming endpoint. Client sends a JSON action payload:

```
{"action": "generate_reply", "contact_id": 123,  
  "email_body": "...", "tone": "professional"}
```

Server streams back tokens one-by-one, ending with [DONE].

7. Security Architecture

Authentication

All REST API endpoints require a valid JWT Bearer token. Tokens are issued on login and verified via the `auth.dependencies` module.

PII Data Sanitization

The `PIISanitizer` utility runs on all context strings assembled by the `ContextBuilder` BEFORE they are sent to the LLM. It uses regex pattern matching to redact:

- Credit Card Numbers -> [REDACTED CREDIT CARD]
- Social Security Numbers -> [REDACTED SSN]
- US Phone Numbers -> [REDACTED PHONE]

Role-Based Access Control (RBAC)

Endpoints for broadcasting WebSocket messages and viewing connection status are protected by admin role checks.

Data Isolation

All database queries are user-scoped (filtered by `user_id`) to ensure complete tenant data isolation.

8. Setup & Installation

Prerequisites

- Python 3.11+
- Redis Server (localhost:6379)
- Ollama (localhost:11434) with a model pulled:
ollama run qwen2.5:1.5b
- PostgreSQL (or SQLite for local development)

Installation Steps

1. Navigate to the backend directory:
cd backend
2. Create and activate a virtual environment:
python -m venv venv
.\\venv\\Scripts\\activate
3. Install all dependencies:
pip install -r requirements.txt
4. Create a .env file with your config:
DATABASE_URL=postgresql://user:pass@localhost/crm_db
REDIS_URL=redis://localhost:6379/0
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=qwen2.5:1.5b
AI_CACHE_TTL_SECONDS=86400
SECRET_KEY=your-jwt-secret-key
5. Apply database migrations:
alembic upgrade head
6. Start the FastAPI server:
uvicorn main:app --reload --host 0.0.0.0 --port 8000
7. Start the Celery worker (separate terminal):
celery -A tasks.celery_app worker --loglevel=info